Version Control (Task-2)

Date: 16-09-2026 Full Name: Arsh Gupta

1. What is Version Control?

Version control is a system that keeps track of changes made to a file or a set of files over time. Every time a change is saved, the system records what was changed, when it was changed, and (if set up properly) who made the change. This means you can always go back and look at an older version of the project, or even restore it completely, without losing any of the work that came after.

Think about how most people handle files without version control: project_final.py, project_final2.py, project_final_ACTUAL.py, project_final_use_this_one.py. This works for a day or two, but it quickly becomes impossible to know which file is correct, what changed between them, or why a change was made in the first place. Version control solves this problem by keeping a single project folder, called a repository, and storing every saved change inside it instead of creating new copies of the whole file.

A few terms come up again and again when talking about version control:

- Repository (repo) - the project folder that is being tracked, along with its full history of changes.
- Commit - a saved snapshot of the project at a certain point in time, along with a short message describing what was done.
- History - the complete list of commits, which together show how the project evolved from the first version to the current one.

For example, in a Mars Rover project, the rover's motor control code might go through several stages: a basic version that just moves the wheels, then a version with obstacle detection added, then a version that fixes a bug in the turning logic. With version control, each of these stages is a separate commit. If the obstacle detection code turns out to be buggy, the team can look at exactly what was added in that commit, compare it with the working version, and fix it or roll it back, instead of rewriting the whole file from memory.

2. Why is Version Control Important?

Version control is not just a convenience, it is close to a necessity once more than one person is working on the same codebase, which is almost always the case in a project like Mars Rover Manipal. Some of the main reasons it matters are given below.

- Tracking changes: Every change made to the code is recorded with a message, so the team can see exactly what was modified and why, instead of guessing based on the code alone.

- Recovering previous versions: If a change breaks something, the project can be rolled back to an earlier, working commit within seconds.
- Collaboration between team members: Multiple people can work on different parts of the project, for example one person on the navigation module and another on the sensor drivers, and later combine their work without overwriting each other's changes.
- Preventing accidental loss of work: Since every commit is saved, work is not lost just because a file was deleted or a laptop crashed, as long as the changes were committed and pushed to a remote repository.
- Branching and experimentation: A team member can try out a new idea, such as a different path-planning algorithm, on a separate branch without touching the main working code. If the idea does not work out, the branch can simply be dropped.
- Maintaining project history: The full history of the project stays available, which is useful when writing reports, debugging old issues, or understanding why a certain design decision was made months ago.
- Identifying who made changes: Each commit is tied to the person who made it, so if a bug is introduced, the team can find out where it came from instead of everyone guessing.
- Helping teams work on large technical projects: A rover project has firmware, control software, simulation code and documentation, often being worked on at the same time by different subsystem teams. Version control keeps all of this organised in one place instead of scattered across everyone's personal folders.

In short, without version control, a team project like Mars Rover Manipal would depend entirely on manual file sharing and everyone remembering to inform each other about changes, which does not scale well once the team and the codebase grow.

3. Types of Version Control

Version control systems can broadly be divided into three types, based on where the history of the project is stored and how it is accessed.

3.1 Local Version Control Systems

This is the simplest form of version control. Here, all versions of the project are stored in a database on the same computer that the user is working on. Every time a file is saved as a new version, it is copied and stored locally, usually as a set of patches or differences between versions.

This works fine for a single person working alone, but it has an obvious problem: there is no way for anyone else to access this history, and if the computer's hard disk fails, the entire history is lost along with it. It is rarely used for real projects today, but it is the basic idea that later systems built on.

3.2 Centralized Version Control Systems (CVCS)

In a centralized system, there is one central server that holds the full repository and its history. Every team member's computer has only a working copy of the current files, not the full history. When someone wants to save a change, they connect to the server and commit directly to it. Systems like SVN (Subversion) and older versions of Perforce work this way.

The advantage here is that everyone can see what everyone else is doing, since it all goes through one place, and administrators have fine control over who can access what. The major drawback is that the central server is a single point of failure. If the server goes down, or if someone loses their internet connection, they cannot commit any changes, and if the server's disk is not backed up, the entire project history can be lost.

3.3 Distributed Version Control Systems (DVCS)

In a distributed system, every team member's machine has a complete copy of the repository, including its entire history, not just the latest snapshot. Git is the most widely used example of this, and there are others such as Mercurial. When someone clones a Git repository, they are not just downloading the current files, they are downloading the full project history.

This has a few clear advantages. Most operations, such as committing, viewing history, or switching branches, happen locally and do not need an internet connection. If the central server (for example, a repository hosted on GitHub) goes down, every team member still has a full backup of the project on their own machine. It also makes branching and merging much faster and more common in day-to-day work. The main downside is that the local copy of the repository can take up more disk space, and there are slightly more concepts to get used to compared to a centralized system.

Mars Rover Manipal, like most modern software teams, uses Git as its distributed version control system, with the shared repository hosted on GitHub so that everyone's work can be pushed to and pulled from one common place.

Type	How it works	Main drawback
Local VCS	Keeps versions of files in a local database on the same machine.	No collaboration, no backup if the machine fails.
Centralized VCS (CVCS)	Single central server holds the repository; clients check out files from it.	Server is a single point of failure; no server access means no work.
Distributed VCS (DVCS)	Every user has a full copy of the repository, including history.	Repository size on disk can be larger; slightly more concepts to learn.

4. Important Git Commands

Git is the version control tool itself, and it works through the command line (or a GUI built on top of it). GitHub, on the other hand, is simply a website that hosts Git repositories online and adds features like pull requests, issues and project boards on top of Git. Git can be used completely offline without ever touching GitHub; GitHub is just one place to host a Git repository, not the only one.

Below are the Git commands that come up most often while working on a team project such as Mars Rover Manipal.

Command	What it does	Example
`git init`	Turns the current folder into a new, empty Git repository by creating a hidden .git folder.	`git init`
`git clone`	Copies an existing remote repository (for example one hosted on GitHub) onto your machine, along with its full history.	`git clone <url>`
`git status`	Shows which files are modified, staged or untracked in the working directory. This is the command you run the most.	`git status`
`git add`	Moves changes from the working directory into the staging area, marking them to be included in the next commit.	`git add rover_control.py`
`git commit`	Saves the staged changes as a new snapshot in the repository's history, with a message describing the change.	`git commit -m "Fix motor speed bug"`
`git log`	Shows the commit history: who committed, when, and the commit message.	`git log --oneline`
`git diff`	Shows the exact line-by-line difference between the working directory and the last commit (or between commits).	`git diff`
`git branch`	Lists existing branches, or creates a new branch if a name is given.	`git branch navigation-module`
`git switch / git checkout`	git switch moves you to a different existing branch. git checkout does the same but is older and also handles restoring files; newer Git versions recommend switch for branches and restore for files.	`git switch navigation-module`
`git merge`	Combines the history of another branch into the branch you are currently on.	`git merge navigation-module`

Command	What it does	Example
`git pull`	Fetches the latest commits from the remote repository and then merges them into the current branch, in one step.	`git pull origin main`
`git push`	Sends your local commits to the remote repository so others can see them.	`git push origin main`
`git fetch`	Downloads new commits and branches from the remote repository but does not merge them into your current branch. It only updates your local copy of the remote's history, so you can review changes before merging.	`git fetch origin`
`git remote`	Manages the links to remote repositories, such as the URL Git should push to or pull from.	`git remote -v`
`git stash`	Temporarily saves uncommitted changes so you can switch branches with a clean working directory, and brings them back later.	`git stash / git stash pop`

A typical day-to-day flow for someone working on, say, the rover's sensor module would look something like this: pull the latest changes with git pull, make edits to the code, check what changed with git status and git diff, stage the changes with git add, save them with git commit, and finally send them to the shared repository with git push. When starting a new feature, it is common to first create a separate branch with git branch and git switch, so that the main branch is not affected until the feature is tested and ready to be merged in with git merge.